

Homework 4: Mapping EER to Relational Schema Diagram

Question 1: Car Parts Manufacturer

EER Diagram Analysis:

PART is the superclass with attributes: Part_no (PK), Description, Manufacture_date.

MANUFACTURED_PART is a subclass with attributes: Batch_no, Drawing_no.

PURCHASED_PART is a subclass with attributes: Supplier_name, List_price.

The 'o' circle indicates an overlapping generalization (a part can be both manufactured and purchased).

Option A: Multiple Relations — Superclass and Subclasses

Create one relation for the superclass and one for each subclass. Subclass relations include the superclass PK as a foreign key.

PART (Part_no, Description, Manufacture_date)

MANUFACTURED_PART (Part_no, Batch_no, Drawing_no)

PURCHASED_PART (Part_no, Supplier_name, List_price)

Relation	Attributes	PK	FK
PART	Part_no, Description, Manufacture_date	Part_no	—
MANUFACTURED_PART	Part_no, Batch_no, Drawing_no	Part_no	Part_no → PART
PURCHASED_PART	Part_no, Supplier_name, List_price	Part_no	Part_no → PART

Option B: Multiple Relations — Subclasses Only

Eliminate the superclass relation. Each subclass relation inherits all superclass attributes. Note: This option works best with disjoint total specialization. With overlapping (o), superclass attributes may be duplicated.

MANUFACTURED_PART (Part_no, Description, Manufacture_date, Batch_no, Drawing_no)

PURCHASED_PART (Part_no, Description, Manufacture_date, Supplier_name, List_price)

Relation	Attributes	PK	FK
MANUFACTURED_PART	Part_no, Description, Manufacture_date, Batch_no, Drawing_no	Part_no	—

PURCHASED_PART	Part_no, Description, Manufacture_date, Supplier_name, List_price	Part_no	—
-----------------------	---	---------	---

Note: Not ideal for overlapping specializations because a part that is both manufactured and purchased would have its common attributes (Description, Manufacture_date) stored redundantly.

Option C: Single Relation with One Type Attribute

Combine everything into one relation with a Type attribute to distinguish subclasses.

PART (Part_no, Description, Manufacture_date, Batch_no, Drawing_no, Supplier_name, List_price, Part_type)

Relation	Attributes	PK	FK
PART	Part_no, Description, Manufacture_date, Batch_no, Drawing_no, Supplier_name, List_price, Part_type	Part_no	—

Part_type $\in \{\text{'Manufactured'}, \text{'Purchased'}, \text{'Both'}\}$. Attributes not applicable to a particular type are set to NULL.

Option D: Single Relation with Multiple Type Attributes

Combine everything into one relation with a Boolean type flag for each subclass.

PART (Part_no, Description, Manufacture_date, Batch_no, Drawing_no, Supplier_name, List_price, Is_manufactured, Is_purchased)

Relation	Attributes	PK	FK
PART	Part_no, Description, Manufacture_date, Batch_no, Drawing_no, Supplier_name, List_price, Is_manufactured, Is_purchased	Part_no	—

Is_manufactured and Is_purchased are Boolean flags. A part with Is_manufactured = TRUE and Is_purchased = TRUE belongs to both subclasses. This is the best single-relation option for overlapping specializations.

Question 2: Car Dealer

EER Diagram Analysis:

VEHICLE is the superclass with attributes: Vin (PK), Price, Model.

CAR (subclass) has: Engine_size. TRUCK has: Tonnage. SUV has: No_seats.

The 'd' indicates disjoint specialization (a vehicle can only be one type).

SALE is a ternary relationship (N:1:1) among VEHICLE (N), SALESPERSON (1), CUSTOMER (1), with attribute Date.

SALESPERSON has: Sid (PK), Name.

CUSTOMER has: Ssn (PK), Name, Address (composite: Street, City, State).

Common Relations (all options)

These relations exist regardless of the generalization mapping option chosen:

SALESPERSON (Sid, Name)

CUSTOMER (Ssn, Name, Street, City, State)

SALE (Vin, Sid, Ssn, Date)

Relation	Attributes	PK	FK
SALESPERSON	Sid, Name	Sid	—
CUSTOMER	Ssn, Name, Street, City, State	Ssn	—
SALE	Vin, Sid, Ssn, Date	Vin	Vin → VEHICLE, Sid → SALESPERSON, Ssn → CUSTOMER

Option A: Multiple Relations — Superclass and Subclasses

VEHICLE (Vin, Price, Model)

CAR (Vin, Engine_size)

TRUCK (Vin, Tonnage)

SUV (Vin, No_seats)

Relation	Attributes	PK	FK
VEHICLE	Vin, Price, Model	Vin	—
CAR	Vin, Engine_size	Vin	Vin → VEHICLE

TRUCK	Vin, Tonnage	Vin	Vin → VEHICLE
SUV	Vin, No_seats	Vin	Vin → VEHICLE

Option B: Multiple Relations — Subclasses Only

Works well here because the specialization is disjoint and total.

CAR (Vin, Price, Model, Engine_size)

TRUCK (Vin, Price, Model, Tonnage)

SUV (Vin, Price, Model, No_seats)

Relation	Attributes	PK	FK
CAR	Vin, Price, Model, Engine_size	Vin	—
TRUCK	Vin, Price, Model, Tonnage	Vin	—
SUV	Vin, Price, Model, No_seats	Vin	—

Option C: Single Relation with One Type Attribute

VEHICLE (Vin, Price, Model, Engine_size, Tonnage, No_seats, Vehicle_type)

Relation	Attributes	PK	FK
VEHICLE	Vin, Price, Model, Engine_size, Tonnage, No_seats, Vehicle_type	Vin	—

Vehicle_type ∈ {'Car', 'Truck', 'SUV'}. This works well for disjoint specialization since each entity belongs to exactly one subclass. Non-applicable attributes are NULL.

Option D: Single Relation with Multiple Type Attributes

VEHICLE (Vin, Price, Model, Engine_size, Tonnage, No_seats, Is_car, Is_truck, Is_suv)

Relation	Attributes	PK	FK
VEHICLE	Vin, Price, Model, Engine_size, Tonnage, No_seats, Is_car, Is_truck, Is_suv	Vin	—

Is_car, Is_truck, Is_suv are Boolean flags. For disjoint specialization, exactly one flag should be TRUE per record. This approach is more suited for overlapping specializations but still works here.